

HumanColonist.java

```
1  package inf112.peripheryplanet.pawns;
2
3  import java.util.List;
4  import java.util.Map;
5
6  import com.badlogic.gdx.Gdx;
7  import com.badlogic.gdx.graphics.Pixmap;
8  import com.badlogic.gdx.graphics.Pixmap.Format;
9  import com.badlogic.gdx.graphics.Texture;
10 import com.badlogic.gdx.graphics.g2d.SpriteBatch;
11 import com.badlogic.gdx.math.Vector2;
12
13 import inf112.peripheryplanet.map.MapCell;
14 import inf112.peripheryplanet.map.PathFinder;
15
16 public class HumanColonist implements Pawn {
17     private Texture image;
18     private Vector2 position;
19     private Vector2 target;
20
21     public HumanColonist(int x, int y) {
22         this.position = new Vector2(x, y);
23         //this.image = new Texture("colonist.png");
24         this.image = new Texture(100, 100, Format.RGB888);
25     }
26
27     public void setTarget(int destX, int destY) {
28         target = new Vector2(destX, destY);
29     }
30
31     public void step(Map<Vector2,MapCell> world) { // do single timestep
32         if (target != null) {
33             if (position.equals(target)) { // are we there yet?
34                 target = null; // yes!
35             } else {
36                 List<Vector2> path = PathFinder.finder(world).from(position).to(target).calculate();
37                 if (path != null) {
38                     // take one step & update map
39                     world.get(position).pawn(null);
40                     position = path.remove(0);
41                     world.get(position).pawn(this);
42                 } else {
43                     // ??? target is unreachable
44                 }
45             }
46         }
47         // ... or do something else
48     }
49
50     public void draw(SpriteBatch batch) {
51         batch.draw(image, position.x, position.y);
52     }
53
54     public Vector2 position() {
55         return position;
56     }
57 }
```

PathFindingTest.java

```
1  package inf112.peripheryplanet.test;
2
3  import static org.junit.jupiter.api.Assertions.assertEquals;
4
5  import java.util.HashMap;
6  import java.util.List;
7  import java.util.Map;
8
9  import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.Test;
11
12 import com.badlogic.gdx.math.Vector2;
13
14 import inf112.peripheryplanet.pawns.HumanColonist;
15 import inf112.peripheryplanet.pawns.Pawn;
16
17
18 public class PathFindingTest {
19
20     private HumanColonist player;
21     private List<Vector2> correctPath = List.of(new Vector2(2, 2),
22         new Vector2(2, 3), new Vector2(2, 4), new Vector2(3, 5));
23     private Map<Vector2,Pawn> world;
24
25     @BeforeEach
26     void setupBeforeEach() {
27         player = new HumanColonist(2, 2);
28         world = new HashMap<>();
29         world.put(player.position(), player);
30     }
31
32     @Test
33     void testPathFinding() {
34         player.setTarget(3, 5);
35         for(Vector2 step : correctPath) {
36             assertEquals(step, player.position());
37             player.step(world);
38         }
39     }
40 }
```